

Arquitetura e Organização de Computadores

Conjunto de Instruções

Características e Funções

Rodrigo Calvo
rcalvo@uem.br

Departamento de Informática – DIN
Universidade Estadual de Maringá – UEM

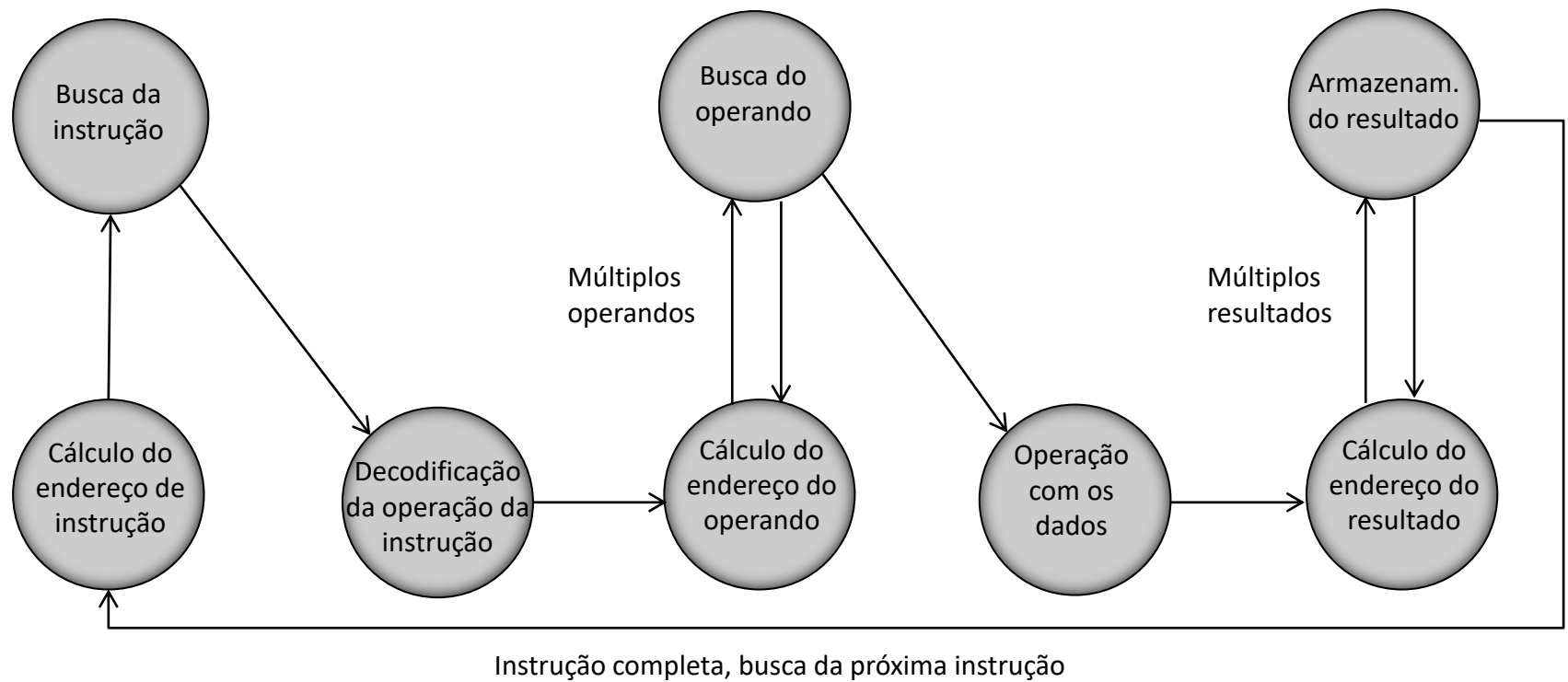
Introdução

- A operação de uma CPU é determinada pelas instruções (comandos) que ela executa, denominadas **instruções de máquina** ou **instruções do computador**.
- A coleção dessas diferentes instruções é chamada **conjunto de instruções**.
- Exemplos:
 - Pentium 2 MMX – C.I. focado em som e multimídia
 - Pentium 3 SSE - C.I. focado em imagem

Elementos de uma instrução

- **Código de operação (*Opcode*):**
 - Especifica o tipo de operação (ex.: ADD, SUB, ...) – código binário.
- **Referência a operando fonte:**
 - Endereço do operando fonte (local em que o operando é lido).
- **Referência a operando de destino:**
 - Endereço do operando de destino (local em que um dado é escrito).
- **Referência à próxima instrução:**
 - Endereço da próxima instrução. Geralmente esta informação é implícita.

Diagrama de estado do Ciclo de Instrução

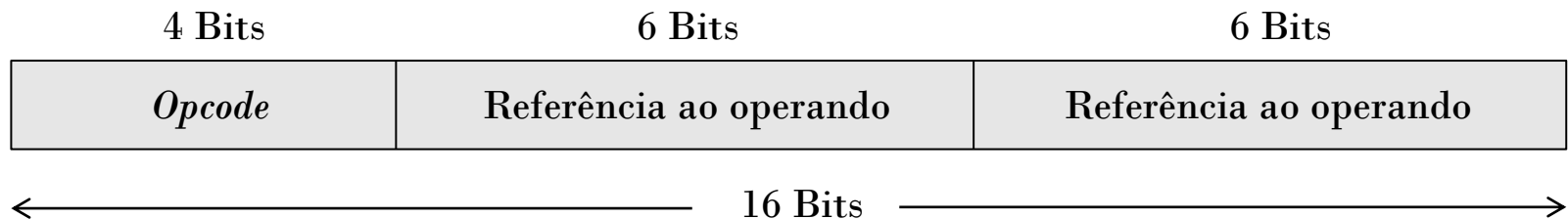


Representação da Instrução

- Em código de máquina, cada instrução tem um padrão de bits exclusivo.
- Para melhor entendimento dos programadores, uma representação simbólica é utilizada.
 - Exemplo: ADD, SUB, LOAD, MUL, DIV, STOR.
- Operandos também podem ser representados desta maneira:
 - ADD A,B.

Formato de instrução simples

- Durante a execução da instrução, o seu *opcode* é lido para um registrador de instrução (IR) no processador para que possa ser identificado.



Tipos de Instruções

- Processamento de dados.
 - Aritmética e lógica.
- Armazenamento de dados (memória principal).
 - Movimentação de dados para dentro ou fora do registrador e/ou locais de memória.
- Movimentação de dados (E/S).
 - Instruções de E/S.
- Controle de fluxo do programa.
 - Instruções de teste e desvio.

Quantidade de Endereços

- 3 endereços:
 - Operando 1, Operando 2, Resultado.
 - Linguagem de alto nível: $a = b + c$
 - Linguagem de baixo nível: `ADD a, b, c`
 - Não é comum.
 - Precisa de palavras muito longas para manter todas as informações.

Quantidade de Endereços

- 2 endereços:
 - Um endereço servindo como operando e resultado.
 - Linguagem de alto nível: $a = a + b$
 - Linguagem de baixo nível: ADD a, b
 - Reduz tamanho da instrução.
 - Requer algum trabalho extra.
 - Necessário armazenamento temporário para manter valor de a.

Quantidade de Endereços

- 1 endereço:
 - Segundo endereço implícito.
 - Normalmente é utilizada para um registrador (acumulador).
 - Comum nas primeiras máquinas.
 - Linguagem de baixo nível: INC a

Quantidade de Endereços

- 0 (zero) endereços:
 - Todos os endereços implícitos.
 - Usa a pilha.
 - Linguagem de alto nível: $c = a + b$.
 - Linguagem de baixo nível:

PUSH a

PUSH b

ADD

POP c

Quantidade de Endereços

Instrução	Comentário
SUB Y, A, B	$Y \leftarrow A - B$
MPY T, D, E	$T \leftarrow D \times E$
ADD T, T, C	$T \leftarrow T + C$
DIV Y, Y, T	$Y \leftarrow Y \div T$

(a) Instruções com três endereços

Instrução	Comentário
MOVE Y, A	$Y \leftarrow A$
SUB Y, B	$Y \leftarrow Y - B$
MOVE Y, D	$T \leftarrow D$
MPY T, E	$T \leftarrow T \times E$
ADD T, C	$T \leftarrow T + C$
DIV Y, T	$Y \leftarrow Y \div T$

(b) Instruções de dois endereços

- Três programas para calcular:

$$f = (a - b) / (c + (d \cdot e))$$

Instrução	Comentário
LOAD D	$AC \leftarrow D$
MPY E	$AC \leftarrow AC \times E$
ADD C	$AC \leftarrow AC + C$
STOR Y	$Y \leftarrow AC$
LOAD A	$AC \leftarrow A$
SUB B	$AC \leftarrow AC - B$
DIV Y	$AC \leftarrow AC \div T$
STOR Y	$Y \leftarrow AC$

(c) Instruções de um endereço

Questão de Projeto: Quantos endereços

- Mais endereços:
 - Instruções mais complexas.
 - Mais registradores de uso geral.
 - Menos instruções por programa.
 - Rapidez na execução de operações
- Menos endereços:
 - Mais instruções por programa.
 - Tempo de execução maior.
 - Programa mais complexo
 - Instruções menos complexas e menores.

Decisões de Projeto

- Repertório de operações:
 - Quantas operações?
 - O que elas podem fazer?
 - Qual a complexidade delas?
- Tipos de dados.
- Formatos de instrução:
 - Tamanho do campo de código de operação.
 - Número de endereços.

Decisões de Projeto

- Registradores:
 - Número de registradores da CPU disponíveis.
 - Quais operações podem ser realizadas sobre quais registradores?
- Modos de endereçamento (mais adiante...).
- RISC vs CISC.

Tipos de Operandos

- Endereços.
- Números:
 - Inteiro/ponto flutuante.
- Caracteres:
 - ASCII (Nada mais são do que números também!).
- Dados lógicos:
 - Bits ou flags.

Tipos de Operações

- Transferência de dados.
- Aritmética.
- Lógica.
- E/S.
- Controle do sistema.
- Transferência de controle.

Transferência de Dados

- Especificam:
 - Origem.
 - Destino.
 - Quantidade de dados.
- Podem ser instruções diferentes para diferentes movimentações.
 - Exemplo: IAS, IBM 370.
- Ou uma instrução e diferentes endereços.
 - Exemplo: VAX, Intel x86.

Aritmética

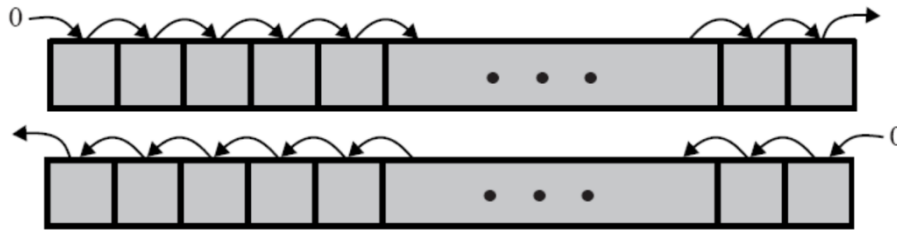
- Envolvem:
 - Transferência de dados antes e/ou depois da operação.
 - Execução da operação na ULA.
 - Atualização dos códigos de condições (*flags*).
- Operações básicas: Soma, Subtração, Multiplicação, Divisão.
- Outras operações possíveis:
 - Incremento ($a++$) ou decremento ($a--$) do operando.
 - Negação do operando ($-a$).
 - Valor absoluto do operando ($|a|$).

Lógica

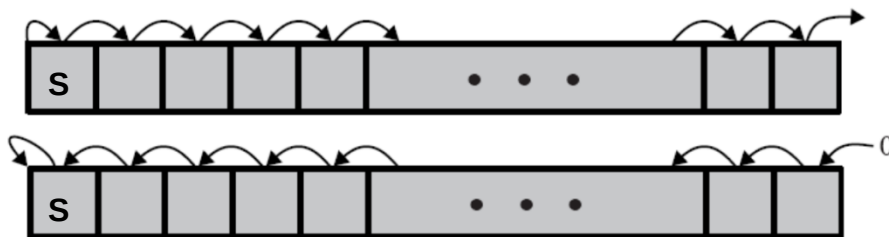
- Operações lógicas básicas sobre dados booleanos ou binários.
- **NOT, AND, OR, XOR:** Mais comuns.
- **EQUAL:** Teste de igualdade binária.
- Deslocamento lógico:
 - Os bits de uma palavra são deslocados para direita ou esquerda.
 - O bit deslocado para fora se perde.
 - Na outra extremidade, um 0 é deslocado para dentro.
- Deslocamento aritmético
 - Agiliza operação com número na representação complemento de dois
 - Deslocamento à direita → divisão por 2, truncando números ímpares
 - Deslocamento à esquerda → multiplicação por 2 (*overflow*)

Operações de Deslocamento e Rotação

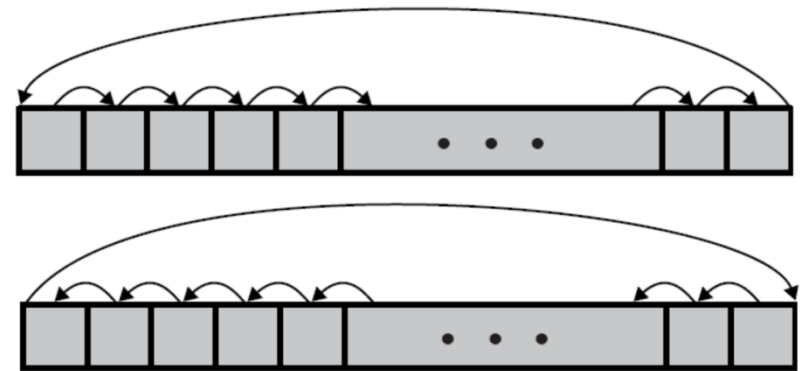
Deslocamento Lógico



Deslocamento Aritmético



Rotação



Entrada / Saída

- Pode ser feito com:
 - Instruções específicas.
 - Usando instruções de movimentação de dados em conjunto com interrupções.
- Exemplo: Intel x86 – Impressão na tela

```
MOV $4, %eax          ; Interrupção de escrita (sys_write)
MOV $1, %ebx           ; Indica modo de escrita (stdout)
MOV hello, %ecx        ; Indica o endereço da string
MOV helloLenght, %edx  ; Indica o tamanho da string
INT 80h                ; Chamada da interrupção
```

Controle do Sistema

- Instruções privilegiadas.
- CPU precisa estar em estado específico: Modo *kernel*.
- Em geral são apenas de uso dos sistemas operacionais.
- Exemplos:
 - Leitura ou alteração de um registrador de controle.
 - Leitura ou alteração de uma chave de proteção da memória.
 - Acesso a blocos de memória usados para controle de processos.

Transferência de Controle

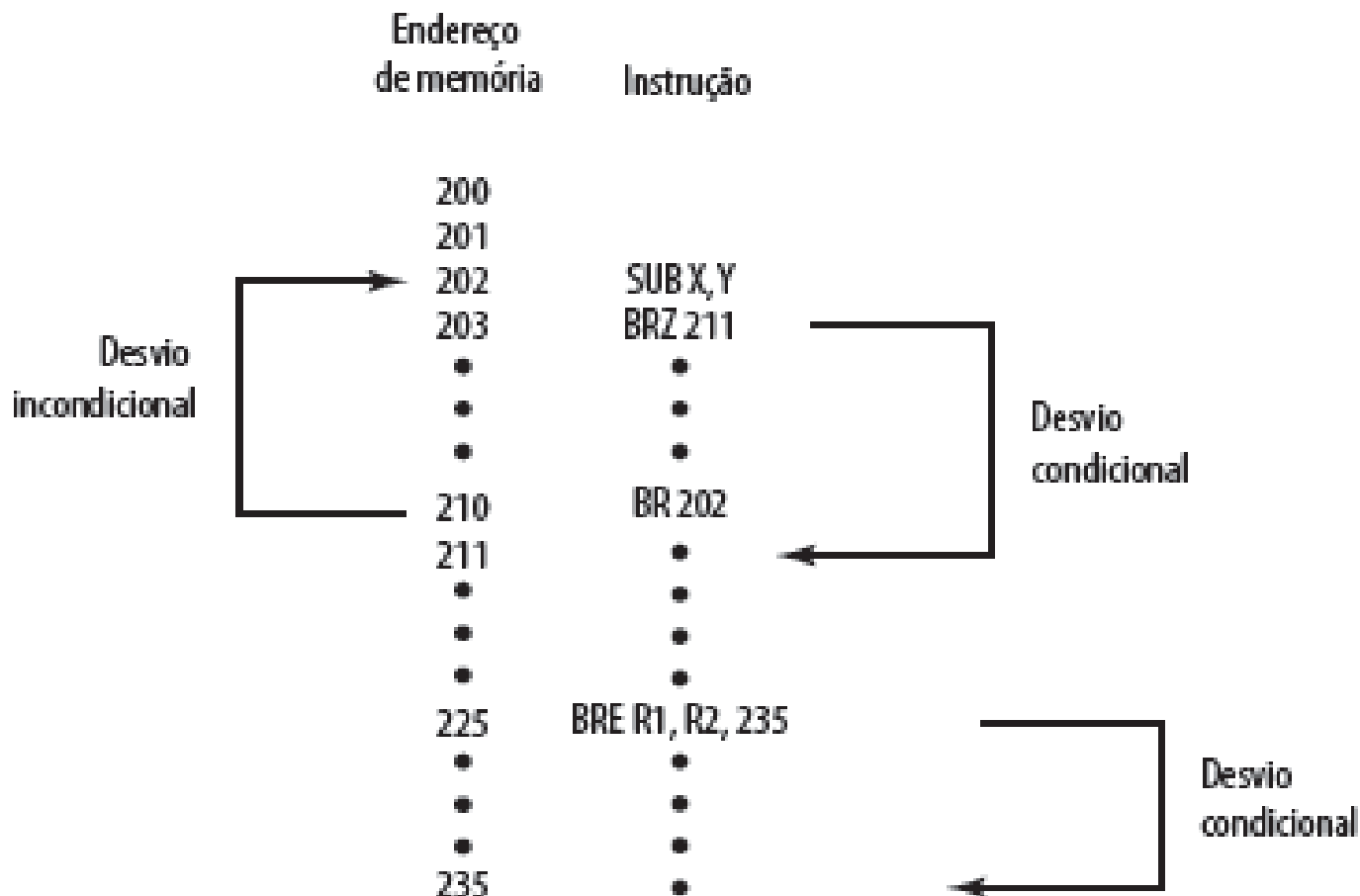
- Muda a sequência de instruções
- O registrador PC é atualizado com o endereço de memória para onde a sequência de instruções foi alterada
- Quando ocorre a transferência de controle?
 - Repetição de instruções (laços de repetição)
 - Tomada de decisão (estrutura condicional)
 - Divisão do problema em subproblemas (subprogramação)

Transferência de Controle

- Desvio: Tem como um de seus operandos o “endereço” da próxima instrução a ser executada.
 - Desvio Condicional: Só é tomado se uma condição for atendida.
 - Desvio Incondicional: É sempre tomado.
- Exemplos:
 - BRZ X Desvio para local X se resultado for zero.
 - BRN X Desvio para local X se resultado for negativo.
 - BRO X Desvio para local X se houver *overflow*.
 - BR X Desvio para local X.

Transferência de Controle

Exemplo de Desvio



Transferência de Controle

- Salto: Inclui um endereço implícito.
- Normalmente implica que uma instrução seja pulada, assim o endereço implícito é igual ao endereço da próxima instrução.
- Exemplo: ISZ – *Increment-and-Skip-if-Zero*.

Memória

301	
...	
309	ISZ R1
310	BR 301
311	ADD R2, R3

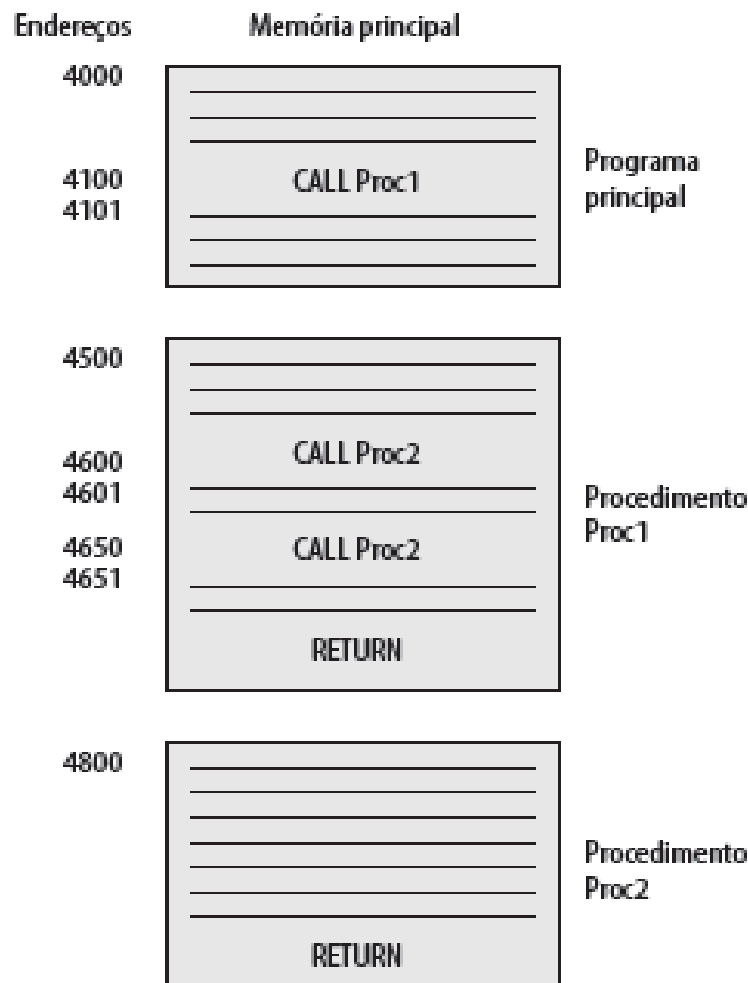
Transferência de Controle

- Chamada de sub-rotina: Em uma chamada de procedimento, o processador é instruído a ir executar o procedimento inteiro e depois retornar ao ponto onde ocorreu a chamada.
- Inovação mais importante no desenvolvimento de linguagens de programação.
- Dois principais motivos:
 - Economia: Elimina repetição do código; e
 - Modularidade: Divisão de grandes tarefas em unidades menores.

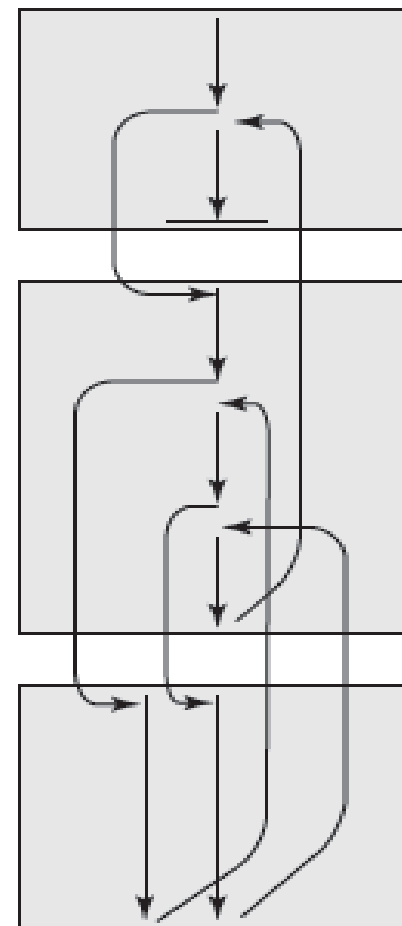
Instruções de Subchamadas de Rotina

- Envolve 2 instruções essenciais:
 - Instrução de chamada (CALL): Desvia a execução da instrução corrente para o início da subrotina.
 - Instrução de retorno (RET): Retorna a execução para o endereço que ocorreu a chamada.

Chamadas de Procedimentos Aninhados



(a) Chamadas e retornos



(b) Sequência de execução

Instruções de Subchamadas de Rotina

- Existem três locais para o armazenamento do endereço de retorno:
 - Registrador;
 - Início do procedimento chamado;
 - Topo da pilha.

Instruções de Subchamadas de Rotina

- Armazenamento do endereço de retorno: **Registrador**

$$RN \leftarrow PC + \Delta$$

$$PC \leftarrow X$$

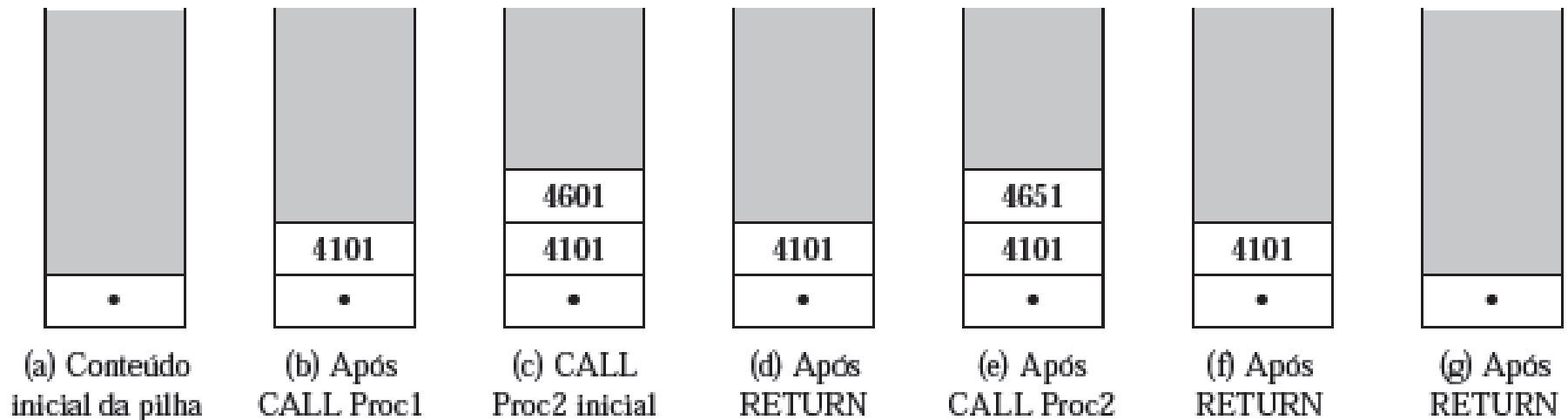
- Armazenamento do endereço de retorno: **Início do procedimento**

$$X \leftarrow PC + \Delta$$

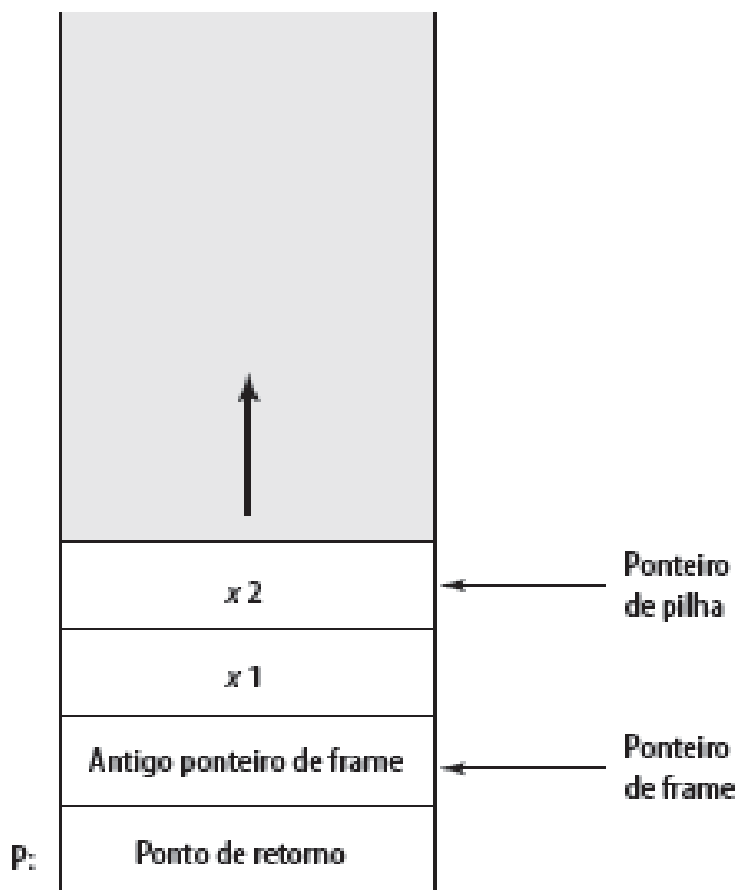
$$PC \leftarrow X + 1$$

Instruções de Subchamadas de Rotina

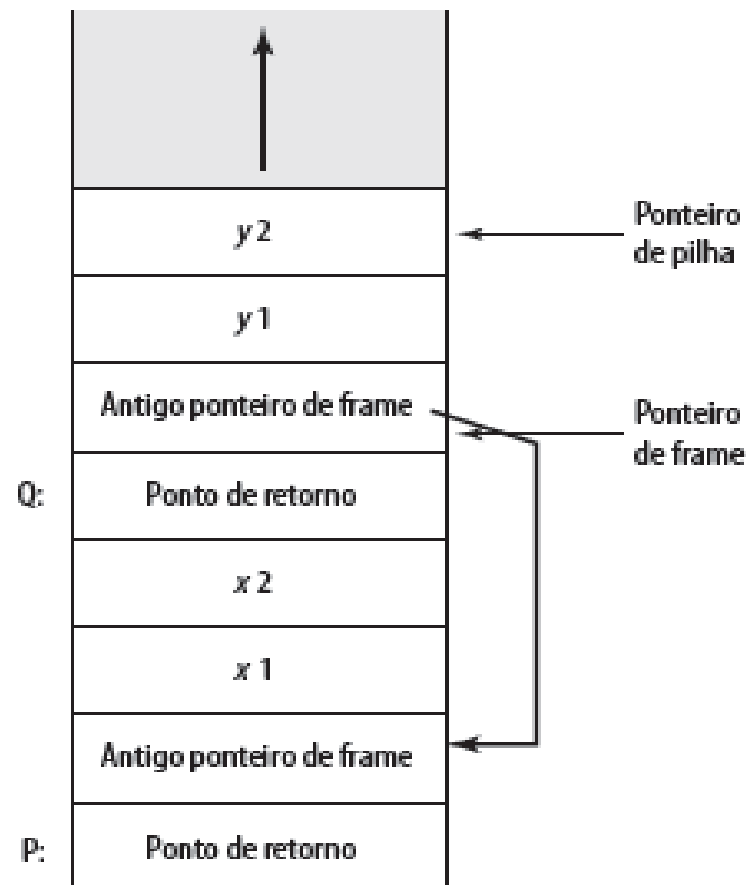
- Armazenamento do endereço de retorno: **Pilha**



Crescimento do frame de pilha



(a) P está ativo



(b) P chamou Q

Comparações de Programas

	Pilha	Registradores gerais	Registrador único
$f = (a - b) / (c + (d \cdot e))$	Push a Push b Subtract Push c Push d Push e Multiply Add Divide Pop f	Load R1, a Subtract R1, b Load R2, d Multiply R2, e Add R2, c Divide R1, R2 Store R1, f	Load d Multiply e Add c Store f Load a Subtract b Divide f Store f
Número de instruções	10	7	8

Operações Comuns do Conjunto de Instruções

Tipo	Nome da operação	Descrição
Transferência de dados	Move (transferência)	Transfere palavra ou bloco da origem ao destino
	Store (armazenar)	Transfere palavra do processador para a memória
	Load (carregar)	Transfere palavra da memória para o processador
	Exchange	Troca o conteúdo da origem e do destino
	Clear (reset)	Transfere palavra de 0s para o destino
	Set	Transfere palavra de 1s para o destino
	Push	Transfere palavra da origem para o topo da pilha
	Pop	Transfere palavra do topo da pilha para o destino
Aritmética	Add	Calcula a soma de dois operandos
	Subtract	Calcula a diferença de dois operandos
	Multiply	Calcula o produto de dois operando
	Divide	Calcula o quociente de dois operandos
	Absolute	Substitui o operando pelo seu valor absoluto
	Negate	Troca o sinal do operando
	Increment	Soma 1 ao operando
	Decrement	Subtrai 1 do operando

Operações Comuns do Conjunto de Instruções

Tipo	Nome da operação	Descrição
Lógica	AND	Realiza o AND lógico
	OR	Realiza o OR lógico
	NOT (complemento)	Realiza o NOT lógico
	Exclusive-OR	Realiza o XOR lógico
	Test	Testa condição especificada; define flag(s) com base no resultado
	Compare	Faz comparação lógica ou aritmética de dois ou mais operandos; define flag(s) com base no resultado
	Set control variables	Classe de instruções para definir controles para fins de proteção, tratamento de interrupção, controle de tempo etc.
	Shift	Desloca o operando para a esquerda (direita), introduzindo constantes na extremidade
	Rotate	Desloca ciclicamente o operando para a esquerda (direita), de uma extremidade à outra

Operações Comuns do Conjunto de Instruções

Transferência de controle	Jump (desvio)	Transferência incondicional; carrega PC com endereço especificado
	Jump conditional	Testa condição especificada; ou carrega PC com endereço especificado ou não faz nada, com base na condição
	Jump to subroutine	Coloca informação do controle do programa atual em local conhecido; salta para endereço especificado
	Return	Substitui conteúdo do PC por outro registrador de local conhecido
	Execute	Busca operando do local especificado e executa como instrução; não modifica o PC
	Skip	Incrementa o PC para saltar para a próxima instrução
	Skip conditional	Testa condição especificada; ou salta ou não faz nada, com base na condição
	Halt	Termina a execução do programa
	Wait (hold)	Termina a execução do programa; testa condição especificada repetidamente; retoma a execução quando a condição for satisfeita
	No operation	Nenhuma operação é realizada, mas a execução do programa continua
Entrada/saída	Input (leitura)	Transfere dados da porta de E/S ou dispositivo especificado para o destino (por exemplo, memória principal ou registrador do processador)
	Output (escrita)	Transfere dados da origem especificada para porta de E/S ou dispositivo
	Start I/O	Transfere instruções para o processador de E/S para iniciar operação de E/S
	Test I/O	Transfere informações de <i>status</i> do sistema de E/S para destino especificado

Bibliografia

- Sttallings, W. **Arquitetura e organização de computadores: projeto para o desempenho**. 8. ed. Prentice Hall, 2009.
- Delgado, J.; Ribeiro, C. **Arquitetura de Computadores**. 2 ed. LTC, 2009.
- Patterson, D. A. ; Hennessy, J.L. **Organização e projeto de computadores – a interface hardware software**. 3. ed. Editora Campus, 2005.